

DhwaniLab — Telugu ASR Benchmarking

Phase 1 report · three pretrained AI4Bharat models on IndicVoices Telugu · WER and OI-WER

1. Objective

Benchmark publicly released Telugu ASR models under a single identical evaluation protocol, and evaluate them with both standard Word Error Rate (WER) and Orthographically-Informed WER (OI-WER), so that reported differences reflect the models rather than the measurement. This is Phase 1; Phase 2 will build a new Telugu dataset and fine-tune these models on it.

2. Experimental setup

Component	Choice
Dataset	ai4bharat/IndicVoices, config telugu, split valid (streamed)
Size	3,295 utterances / 37,350 reference words
Reference field	normalized
Audio	mono, resampled to 16 kHz
Language model	none, for any of the three models
Output	per-utterance CSV: index, speaker_id, duration, reference, prediction, S/D/I

The validation split is the only evaluation split IndicVoices publishes; there is no separate test split (train = 269,300 utterances, valid = 3,295).

Models evaluated

- **IndicWhisper** (whisper-medium-te_alldata_multitgu) — standard HuggingFace checkpoint, decoded autoregressively with language=te, task=transcribe.
- **IndicConformer** (ai4bharat/indic-conformer-600m-multilingual) — gated repository, loaded with trust_remote_code; resolves to an ONNX Runtime model, decoded with greedy CTC.
- **IndicWav2Vec** (te.pt, 3.53 GB Fairseq checkpoint) — no HuggingFace Telugu weights exist, so it runs in an isolated Python 3.10 / torch 1.13 virtual environment via a subprocess decoder, with audio pre-dumped to disk. Greedy CTC (Viterbi), no language model.

3. How WER is computed

3.1 Per utterance

An *utterance* is one audio clip with one reference transcript — the unit the dataset ships and the unit each model transcribes. Each utterance is scored independently. Both the reference and the prediction are whitespace-normalized and split into word tokens. The two token sequences are then aligned using **minimum edit distance** (word-level Levenshtein alignment), which finds the alignment requiring the fewest edits. Each aligned position is classified as:

- **Hit** — reference word and predicted word are identical.
- **Substitution (S)** — a reference word was replaced by a different predicted word.
- **Deletion (D)** — a reference word has no counterpart in the prediction (word missed).
- **Insertion (I)** — a predicted word has no counterpart in the reference (word invented).

3.2 Corpus aggregation

Per-utterance counts are summed across all 3,295 utterances *before* dividing. The reported figure is therefore a corpus-level rate:

$$\text{WER} = (\sum S + \sum D + \sum I) / \sum N$$

where N = number of reference words in an utterance

This is deliberately not the mean of per-utterance WERs. Averaging per-utterance rates gives a one-word utterance the same weight as a forty-word utterance, which inflates the influence of short utterances and produces numbers that are not comparable with published results. Per-utterance WER is still stored in the CSVs, but it is used only for error analysis, never for the headline figure.

3.3 Ensuring the three models are comparable

- All three models were run over the same streamed split, so utterance *index* aligns across the result files.

- References were compared position by position across the runs: **3,295 of 3,295 matched exactly**. Had they differed, the WERs would not have been measuring the same task.
- All three are scored against the identical denominator of 37,350 reference words.
- A separate notebook recomputes the WERs from the raw stored predictions; it reproduced the original IndicWhisper figure exactly, confirming the harness itself introduces no difference.

3.4 One denominator subtlety

The references contain 62 <unintelligible> transcriber markers. Removing them — which OI-WER must do, since no model can produce them — drops 61 tokens, taking the denominator from 37,350 to **37,289**. Every OI-WER figure in this report is therefore compared against a **normalized-text WER computed on that same 37,289-word denominator**, not against the raw 37,350-word figure. Comparing across two different denominators would misattribute part of the change to the metric.

4. How OI-WER is computed

OI-WER addresses a known problem in Indian-language ASR evaluation: many predictions marked wrong are in fact legitimate alternative spellings. The published method (arXiv:2603.00941) replaces each single reference word with a **set of acceptable variants**; a prediction matching any variant is counted as a hit rather than a substitution.

4.1 Algorithm

The reference is represented as an ordered sequence of *segments*, each carrying one or more accepted token sequences. Alignment then runs as a dynamic program over (segment index, prediction position), where each segment may be matched by any of its variants — and a variant may span multiple words, so one reference word can legitimately match two predicted words and vice versa. The error counting and corpus aggregation are otherwise identical to Section 3, and **the denominator remains the reference word count**, so WER and OI-WER stay directly comparable.

4.2 Where the variants come from

Two independent sources are used, and the report separates their contributions rather than reporting only the combined figure.

Source A — deterministic rules

- **Inverse Text Normalization (ITN)** — spans of Telugu number words are parsed to their numeric value and the digit form is added as an accepted variant, so ఎనిమిది and 8 match. The parser handles units, teens, tens, and the multipliers hundred / thousand / lakh / crore, including compound values such as 399,999.
- **Compound merging and splitting** — a reference token matches a group of predicted tokens (or the reverse) when their concatenations are identical. This is pure string equality, requiring no lexicon, and handles Telugu agglutination.
- **Annotation tag removal and surface normalization** — <unintelligible> markers are removed; Unicode NFC, punctuation and symbol stripping, and case folding for embedded Latin text are applied.

Source B — a local LLM (gemma3n:e2b via Ollama)

The paper generates variants with Gemini-2.5-Pro and then has native speakers review them. No LLM API was available for this project, so a local model small enough for 5 GB of VRAM was run over all 3,295 references, prompted with AI4Bharat's own expert-verified Telugu guideline examples, taken from their published repository.

A small local model is not a substitute for Gemini plus human review, and it behaved accordingly: alongside genuine spelling variants it produced translations, synonyms, and invented words. This matters more than it might appear, because **a bad variant can only ever make a score look better** — the alignment takes the minimum over variants, so an unfiltered hallucination silently forgives a real error. Every LLM-proposed variant is therefore passed through a filter before it can affect any score:

- **Anchoring.** A proposed group is discarded entirely unless it contains the reference word itself, copied exactly. This drops groups where the model rewrote or invented the original, and it keeps the reference word count fixed, so the denominator cannot drift.
- **Consonant skeleton.** Telugu vowel signs, virama and anusvara are stripped from both strings; the remaining consonant skeletons must agree within one edit. Spelling variation preserves the skeleton; substituting a different word does not.
- **Short-word guard.** For anchors of four characters or fewer the skeletons must match *exactly* — only vowel signs may differ. On short words a single edit often lands on a different real word, which neither of the tests above can detect.
- **Edit ratio.** Character edit distance must be within 34% of the reference word's length, or the two must be identical once spaces are removed (which admits compound splits and merges).

The short-word guard was added after inspecting the accepted variants directly. Four pairs — ఈ/ఏ, మ్మీ/నీ, అదీ/అదీ, అండీ/అంటీ — were among the *most frequent* accepted variants and were each forgiving genuine recognition errors on very common words. Adding the guard removed 1,588 of 6,765 accepted variants (23%) and moved OI-WER **up** by 0.05 to 0.12 points. Every figure in this report is post-guard.

Worked examples, all taken from actual gemma3:4b output on this dataset: రీచ్ → రీచ్ is accepted (same skeleton, one edit); తొలంగణల్ → తొలంగణ ల్ is accepted (compound split); కవడనీకి → చేయడనీకి is rejected (different word); వీల్సెన్ → వీరదేశలు is rejected (translation); నేను → న is rejected (different word).

LLM proposal stage	Count
Word groups proposed by gemma3n:e2b	37,069
Groups anchored to a reference span	35,008
Alternative spellings offered	4,871
Alternatives that passed the filter	3,940
Reference spans actually augmented	3,900

The filter rejected 19% of what the model proposed.

4.3 Choosing the variant generator

Two local models were run over the identical 3,295 references, with the identical prompt and the identical filter. The smaller one won on every ASR model.

	gemma3:4b	gemma3n:e2b
Word groups proposed	36,613	37,069
Groups anchored (kept the original word)	32,782 (89.5%)	35,008 (94.4%)
Alternatives offered	12,247	4,871
Alternatives kept	5,177	3,940
Rejected by the filter	58%	19%
OI-WER, IndicConformer	26.128%	25.600%
OI-WER, IndicWav2Vec	51.763%	51.420%
OI-WER, IndicWhisper	53.509%	53.260%

gemma3n:e2b has roughly half the effective parameters, proposes 2.5 times fewer variants, and has them rejected three times less often. It is more conservative and more often right; it also mangles the reference less, anchoring 94.4% of groups against 89.5%. Its unique contributions are genuine Telugu orthography — చేయడము → చేయడం, వళిలకి → వళిళి, ఇంటర్నెట్ → ఇంటర్నెట్ — whereas gemma3:4b's most frequent unique contributions were the short-word confusions that prompted the guard above.

A practical note for anyone reproducing this: gemma3n:e2b is slower despite being smaller. Its per-layer-embedding design parks 3.84 GB in pinned host memory rather than VRAM, so tokens stream over PCIe. The Ollama log line “31/31 layers offloaded” is misleading; the CUDA0 versus CUDA_Host buffer sizes are the figures that matter.

5. Results

5.1 Headline

Model	WER	OI-WER	Gain	Decoding
IndicConformer	27.84%	25.60%	-2.24	greedy CTC
IndicWav2Vec	54.58%	51.42%	-3.16	greedy CTC
IndicWhisper	56.76%	53.26%	-3.50	autoregressive

WER here is the normalized-text figure on the 37,289-word denominator (Section 3.4). Ranking is identical under both metrics: IndicConformer < IndicWav2Vec < IndicWhisper — OI-WER reorders nothing.

5.2 Rules against LLM

Because the two variant sources are independent, each was scored alone and then together.

Configuration	IndicConformer	IndicWav2Vec	IndicWhisper
WER, raw text (37,350 words)	27.84%	54.56%	56.74%
WER, normalized (37,289 words)	27.84%	54.58%	56.76%
OI-WER, rules only	26.93%	52.48%	54.34%
OI-WER, LLM variants only	26.07%	52.69%	54.82%
OI-WER, rules + LLM	25.60%	51.42%	53.26%

Substitutions removed relative to the normalized-text baseline:

Variant source	IndicConformer	IndicWav2Vec	IndicWhisper
Rules only	184	486	618
LLM only	511	530	584
Rules + LLM	612	803	973

The improvement is almost entirely in substitutions, which is the expected signature: accepting a variant can only convert a wrong word into a correct one.

5.3 What the variants corrected

- **The digit penalty was entirely one-sided.** ITN removed 124 substitutions from IndicWhisper and zero from IndicConformer, because IndicConformer never emits digits while IndicWhisper does so in 129 utterances. Cases such as reference 'eight' against prediction '8' were perfect recognitions previously scored as complete errors.
- **Merge/split was the larger rule effect**, as expected for an agglutinative language: a single Telugu word written by the model with an internal space was previously counted as two separate errors.
- **The LLM contributed a different class of correction** — matra and vowel-length differences and loanword spellings, which no string rule can produce without a lexicon. This is the category the rules cannot reach, which is why the two sources are largely additive.

Representative corrections — reference against prediction, identical words differing only in spacing, number format, or vowel length:

ప్రదర్శనదీ / ప్రదర్శనదీ కి · మాడువేల / మాడు వేల · ఎనిమిది / 8 · రీచ్ / రిచ్

6. Verification of the scoring code

The OI-WER engine was written from scratch, so it was validated before being trusted.

6.1 Against the authors' own implementation

AI4Bharat have since released their reference scorer. It is vendored into this repository (oiwer/vendor/oiwer_core.py, MIT) and both engines were run over the identical inputs.

Configuration	Model	Our engine	AI4Bharat scorer
no variants	IndicConformer	27.845%	27.845%
no variants	IndicWav2Vec	54.576%	54.576%
no variants	IndicWhisper	56.762%	56.762%
rules + LLM	IndicConformer	25.6%	26.144%
rules + LLM	IndicWav2Vec	51.42%	52.788%
rules + LLM	IndicWhisper	53.26%	54.638%

With variants switched off the two engines agree to the digit on all three models — the strongest available evidence that our alignment and counting are correct. With variants enabled our figures are slightly lower, because the AI4Bharat scorer greedily commits to one multi-word variant before running its dynamic program, while ours searches all variants inside the program and is therefore optimal by construction.

6.2 Other checks

- With trivial single-variant segments the engine reduces to standard WER and reproduces both published figures to six decimal places (27.839357% and 56.736278%), matching the independent jiwer computation.

- Nine unit tests cover exact match, substitution, deletion, insertion, multi-word variants, numeral variants, and a negative case confirming a genuine error is still counted as an error.
- Eleven further unit tests pin down the LLM variant filter, including the specific hallucinations observed in gemma3:4b output — a translation, a synonym, a different word, an invented extra group — each asserted to be rejected, and genuine matra and compound-split variants asserted to be accepted.
- The invariant $OI-WER \leq WER$ was verified to hold for every configuration.

7. Interpreting the model gap

- **Domain match.** IndicConformer is AI4Bharat's own model, trained on IndicVoices, so this validation split is in-domain for it; IndicWhisper was fine-tuned on a different Telugu mixture and faces a domain shift. The supportable claim is that IndicConformer is stronger *on this benchmark*, not that it is a better Telugu ASR model in general.
- **Decoding asymmetry.** IndicConformer and IndicWav2Vec used greedy CTC with no language model; IndicWhisper generates autoregressively with an implicit internal language model.
- **Different failure modes.** IndicWhisper's much higher deletion and insertion counts indicate dropped content and hallucinated repetition on short utterances, rather than uniform mis-recognition.
- **OI-WER helps the weaker models most.** The gain grows with the error rate, which is what the metric is for: a model that is merely spelling things differently is penalised more heavily by plain WER the more output it produces.

8. Limitations

- **This is still an approximation of OI-WER, not a reproduction of it.** The published method pairs Gemini-2.5-Pro with review by 61 native speakers across seven variation categories. Ours pairs hand-written rules with an unreviewed 4B local model behind a conservative filter. The filter is deliberately strict, so it will reject genuine variants as well as bad ones; our figures should be read as a **lower bound** on the true OI-WER gain.
- **No native-speaker review has taken place.** Neither the Telugu number lexicon nor the accepted LLM variants have been checked by a Telugu speaker. This is the single most valuable next step, and it is cheap: the accepted variants are stored and can be reviewed as a flat list.
- **The LLM variants are unvalidated against a gold set.** We can show what the filter rejects, but without AI4Bharat's own annotations we cannot measure what fraction of true variants it misses.
- **No human-perceived WER reference.** The paper validates OI-WER against post-edited human transcripts; we have no such reference, so we cannot measure how much closer our metric sits to human judgement.

9. Note on IndicWav2Vec decoding

AI4Bharat never published Telugu IndicWav2Vec weights in HuggingFace format — the hosted repository is empty — so the only Telugu artifact is a Fairseq checkpoint tied to Python 3.10 and torch 1.13. It was evaluated by running that stack in an isolated virtual environment, driven as a subprocess, with audio pre-dumped to disk so the old environment never touches the modern data libraries. The run completed all 3,295 utterances at roughly 8 utterances per second.

One caveat matters for comparison with published figures: AI4Bharat's official IndicWav2Vec pipeline decodes with KenLM and a lexicon. That was deliberately skipped here so that all three models are compared without an external language model. Our figure is therefore LM-free and should not be read against published numbers obtained with LM decoding.

A decoding detail worth recording: in Fairseq's CTC convention the blank symbol is the dictionary's bos index, not pad. Filtering the wrong index leaves every blank frame in the output, which renders as literal markup and produces a near-100% error rate that looks like model failure rather than a decoding bug.

10. Next steps

- Native-speaker review of the accepted LLM variants and the Telugu number lexicon — the cheapest available improvement in confidence.
- Confirm whether AI4Bharat's orthographically-informed IndicVoices benchmark (Telugu: ~2.6K utterances, 92.7K variations, native-speaker reviewed) can be obtained directly — this would replace our approximation with the reference annotations.
- Error analysis from the stored predictions: WER against utterance duration, per-speaker breakdown, substitution patterns, and CER alongside WER. No GPU inference required.
- Optionally re-run IndicWav2Vec with KenLM + lexicon decoding to match AI4Bharat's official pipeline.
- Phase 2: Demucs source separation, VAD segmentation, YouTube subtitles, MFA alignment, a 10-hour Telugu dataset, and fine-tuning all three models — re-evaluating with the same WER and OI-WER harness so the improvement from fine-tuning is measurable per model.

